Journal of Scheduling (2025) 28:289–310
https://doi.org/10.1007/s10951-025-00836-1

A solution framework for multi-skilled project scheduling problems with hierarchical skills

Jakob Snauwaert[1] · **Mario Vanhoucke**[1,2,3]

Accepted: 12 January 2025 / Published online: 23 March 2025
© The Author(s) 2025

Abstract
Multi-skilled project scheduling concerns the assignment of multi-skilled resources to activities and the scheduling of these activities in order to minimise the project makespan. Since the resources in these problems can be discerned based on their mastered categorical skills, they are considered to be individual entities (human beings) rather than a general class or type of resources. Therefore, researchers have been looking into multi-skilled resources to investigate which other characteristics differentiate them from one another. A main line of research in the last years studies the incorporation of hierarchical skills and their impact on the decision-making in scheduling problems. Hierarchical skills indicate the level at which resources can perform their different skills. In this paper, we present six multi-skilled resource-constrained project scheduling problems with hierarchical skills. In each of these problems, the hierarchical skills have a different impact on the project and its objectives. Solutions are constructed using a solution framework where new problem-specific local searches are embedded in a genetic algorithm. This new framework is tested on existing data from the literature. Computational experiments compare the quality of the solution approaches and the local searches to an integer programming solver. Furthermore, the principal features and tendencies of each problem are discussed. Finally, best-known solutions and lower bounds for all presented problems will be provided.

Keywords Resource-constrained project scheduling · Skills · Genetic algorithm · Local searches

1 Introduction

The topic of this paper is situated in the field of project management. In academic terms, project management is part of operational research, which mainly concentrates on the optimisation of operational problems. More specifically, we focus on the subfield of project scheduling, which can be defined as setting the blueprint for how a project should be realised. The research that will be conducted revolves around project scheduling with resource constraints. This has been widely studied in the literature in the form of the resource-constrained project scheduling problem (RCPSP), which is NP-hard (Blazewicz et al., 1983). The RCPSP deals with scheduling activities and assigning resources to activities with, in most cases, cost or time minimisation as an objective. This paper regards resources as individual human beings and investigates how they can be optimally employed dependent on their impact on the project. Since we also introduce skills in these problems, the main focus lies on the multi-skilled resource-constrained project scheduling problem (MSRCPSP), first presented by Néron (2002).

The MSRCPSP is a subproblem in project scheduling that intends to minimise the makespan of projects by making scheduling and assignment decisions for resources that master several different skills. In the classic version of the RCPSP, workers are assumed to be identical and implicitly master the complete skillset, whereas the workforce in the multi-skilled RCPSP has a heterogeneous skill-mix. In the MSRCPSP, multi-skilled workers do not have the skills to work at every activity of the project. In comparison with single-skilled resources, i.e. workers that master only one

✉ Mario Vanhoucke
mario.vanhoucke@ugent.be

Jakob Snauwaert
jakob.snauwaert@ugent.be

1 Faculty of Economics and Business Administration, Ghent University, Tweekerkenstraat 2, Ghent 9000, Belgium

2 Operations and Technology Management Centre, Vlerick Business School, Reep 1, Ghent 9000, Belgium

3 UCL School of Management, University College London, 1 Canada Square, London E14 5AA, UK

skill, multi-skilled resources can be assigned to a broader range of tasks and are therefore more flexible. This flexibility increase, together with other advantages, depends on the size and characteristics of the pool of multi-skilled resources in the project. In this paper, we consider the extension of heterogeneous efficiencies, which means that the resources will master the skills at a certain hierarchical skill level. Resources are therefore characterised by two concepts: categorical skills and hierarchical skills (De Bruecker et al., 2015). A *categorical skill* refers to the understanding of a resource for the different components of the project and recognising how the different components fit together. In a project environment, the categorical skills are perceived as the amount of skills an employee masters. A *hierarchical skill*, in turn, specifies the deeper knowledge and understanding as well as a higher proficiency an employee can have for a certain skill. Consequently, hierarchical skills are translated into a set of different efficiency levels, i.e. the level of a skill defines how proficient the resource is when performing the skill. In most cases, hierarchical skills have a more direct impact on the project objectives, while the categorical skills have a more indirect influence. On the one hand, a higher hierarchical skill level can have an impact on the speed of execution of a resource and can therefore reduce activity durations (Tiwari et al., 2009). On the other hand, rather than having an impact on the activity duration, larger number of categorical skills will result in more flexibility in assigning resources to activities, which can increase the number of scheduling possibilities that can reduce the makespan (Eiselt and Marianov, 2008).

Since organisations are always striving for a diversified workforce that can successfully complete their projects within the deadline and budget, a good balance of older, more experienced employees and younger, more dynamic and creative employees is paramount in today's organisational structure (Saxena, 2014). Equally important is the inclusion of a set of specialist workers that counterbalance the large group of regular workers. Clearly, differences in skill-level and heterogeneous efficiencies will characterise these workforces. The focus of this paper will be on analysing the characteristics of robust multi-skilled workforces in which the skill levels of workers have different types of impact on the project scheduling decisions. We will focus on team composition to fully exploit the benefits of skills, solve recent problems from the literature using modern optimisation approaches and integrate categorical and hierarchical skills into the existing set of algorithms. Additionally, we will perform large computational experiments on existing artificial data to evaluate the influence of skills on projects.

Throughout this paper, we will use the abbreviation hMSRCPSP for the MSRCPSP with hierarchical skills. As mentioned before, the introduction of hierarchical skills can have a plethora of different effects on the project schedule or resource assignment. The effects that will be researched in this paper are defined by the problems in Snauwaert and Vanhoucke (2022). In their paper, a total of six multi-skilled project scheduling problems with hierarchical skills are presented together with multiple linear programming formulations. A more detailed description of these problems can be found in Sect. 5.

- hMSRCPSP with variable activity durations: this problem deals with variable activity durations due to efficiency variations of the resources.
- hMSRCPSP with task-level restriction: in this problem, each activity requires resources with a specific skill level to be assigned.
- hMSRCPSP with resource-level flexibility: the activities in this problem require resources with a mastered skill level higher than or equal to the required level.
- hMSRCPSP with cost of resource skills: this problem minimises the total resource cost in the project given a set of skill costs.
- hMSRCPSP with heterogeneous resource usage: this problem assumes that resources with higher hierarchical skills are more efficient at using a common good. The goal is to minimise the usage cost of this common good.
- hMSRCPSP with quality as rework probability: in this problem the rework probability in the project is minimised, which is determined by the hierarchical level of the resources.

The contribution of this work lies mainly in the development of problem-specific solution procedures to provide new best-known solutions for the six problems that have been recently published in the literature. For these problems, mathematical formulations and state-of-the-art data are available, but no advanced solution algorithms have been presented, neither have any been tested on this new data. Therefore, the results of the solution procedures in this paper can be used as a benchmark for future research. The found solutions are made publicly available on our website in order to allow fair comparisons and future improvements to the presented procedures (https://www.projectmanagement.ugent.be/research/project_scheduling/MSRCPSP). We present detailed computational results and analyse different aspects and pitfalls of each problem with regard to the impact of categorical skills, hierarchical skills and total workforce composition on the selected objective and subsequent decisions.

The paper outline is as follows. Section 2 presents the literature on the MSRCPSP and hierarchical skills. Section 3 defines the multi-skilled resource-constrained project scheduling problem. Section 4 describes the genetic framework on which the solution approaches of this paper are based. Section 5 explains the various extensions with hierarchical skills and shows the problem-specific solution

algorithms that are developed. Section 6 displays a large number of computational experiments that were performed and comprehensively explains their results. Finally, in Sect. 7 the main conclusions of this paper are set out.

2 Related literature

The project scheduling literature and more specifically the RCPSP and variations of the RCPSP have been studied extensively in the last decades. We refer the reader to the following studies for more information: Brucker et al. (1999), Demeulemeester and Herroelen (2006), Hartmann and Briskorn (2010) and Hartmann and Briskorn (2021). In the following paragraphs, the literature on the MSRCPSP and its extensions is set out.

MSRCPSP Although the research on resource-constrained project scheduling is already well established in the literature, the introduction of skills is only a recent development. A project scheduling problem with resources that were characterised by skillsets originated in Néron (2002). With a published clear definition of the MSRCPSP, a handful of other researchers were interested in the multi-skilled extension of the RCPSP and they developed heuristic and exact algorithms for this problem. Bellenguez-Morineau and Néron (2007) presented a branch-and-bound, with a branching scheme that concentrated on slack reduction of the activities. A mixed-integer linear programming formulation with some valid inequalities and a two-phase heuristic are suggested in Correia et al. (2012). Correia and Saldanha-da Gama (2014) were the first to investigate the MSRCPSP with a cost objective. Their costs were split into a fixed and variable component and depended on the number of categorical skills a resource possesses. Montoya et al. (2014) developed a column generation approach embedded in a branch-and-price procedure. Almeida and Correia (2018) created a biased random-key genetic algorithm with parallel and serial scheduling scheme alternatives. In Almeida et al. (2019), seven continuous and time-indexed linear programming formulations are set out for the MSRCPSP alongside a comparison of the lower bounds supplied by their linear relaxations. Snauwaert and Vanhoucke (2023) propose an extension to the classification scheme of Herroelen et al. (1999) for the MSRCPSP and its variations. Additionally, new artificial and empirical data instances are presented that are based on new skill and resource parameters. For an overview of skills in the scheduling literature, we refer the reader to Afshar-Nadjafi (2021).

Extensions The MSRCPSP has also been researched under additional constraints that are prevalent and established in the research on the RCPSP. Drezet and Billaut (2008) constructed a greedy heuristic and a Tabu search algorithm for the MSRCPSP with activity pre-emption, deadline restrictions, daily working limitations and a lateness objective. A multi-skilled project scheduling problem with different execution modes is studied in Maghsoudlou et al. (2016). The MSRCPSP with pre-emption and due date intervals is investigated in Maghsoudlou et al. (2019), which was solved by a hybrid ant colony algorithm with local searches. Polo-Mejía et al. (2019) describe a priority-list-based heuristic for a project scheduling problem with multi-skilled resources with variable availability, deadlines and pre-emption. Multi-skilled resources have also featured abundantly in multi-project scheduling problems. Chen et al. (2020) study a project portfolio problem with multi-skilled resources under quality, learning and makespan objectives, for which they developed a Pareto ant colony algorithm. Yu et al. (2022) provide a two-stage heuristic with softmax scoring mechanisms for the multi-project scheduling problem with multi-skilled staff sharing. Haroune et al. (2022) consider a two-agent MSRCPSP with multiple projects that is solved through a hybrid approach that combines different solution methods.

Hierarchical skills Besides the literature on the regular MSRCPSP, a recent surge in the literature on variations and extensions has occurred. Significantly, an array of papers that include hierarchical skills of resources in scheduling problems have been published. The first to consider the hMSRCPSP were Bellenguez and Néron (2004) who suggested a lower bound based on energetic reasoning. Multiple papers investigated a scheduling problem in which the hierarchical levels had an impact on the efficiency of the multi-skilled resources, which resulted in a variation of the activity duration (Valls et al., 2009; Javanmard et al., 2017; Dang Quoc et al., 2020; Snauwaert and Vanhoucke, 2021; Hosseinian and Baradaran, 2021b). In some studies, the hierarchical skills specify the effectiveness of a resource in a project. In that case, the skill levels do not necessarily have a direct impact on the activities in the project, rather in these types of problems the goal is to maximise the total effectiveness of the workforce (Yannibelli and Amandi, 2011; Zabihi et al., 2019). Hierarchical skills can be incorporated into the MSRCPSP as a restriction for assigning resources to activities. The skill requirements in these problems now also specify a minimum level that is required to execute the activity. As such, resources that have a lower hierarchical level than specified are not allowed to work on the activity. One of the first papers to discuss this problem was Myszkowski et al. (2015), who created an ant colony metaheuristic for this problem. Additionally, Myszkowski et al. (2015) described a new dataset for this problem which spawned an abundance of other research papers that developed methodologies for this problem (Zheng et al., 2017; Myszkowski et al., 2018; Wang and Zheng, 2018; Laszczyk and Myszkowski, 2019; Zhu et al., 2019; Lin et al., 2020; Zhu et al., 2021; Li et al., 2022; Myszkowski and Laszczyk, 2022). A version of this problem with supplementary skill switch times (or transfer times) has

been recently published in Tian et al. (2022) and Hosseinian and Baradaran (2021a) who, respectively, proposed a multi-objective evolution strategy metaheuristic and a multi-agent optimisation algorithm. Resources with higher hierarchical skill levels can also have a positive impact on the use of electricity or energy in projects by being more experienced and economical. In these problems, the goal is to minimise the total usage of electricity which is determined by introducing time-of-use tarifs (Najafzad et al., 2019; Maghsoudlou et al., 2021; Javanmard et al., 2022). A different problem is researched in Maghsoudlou et al. (2017), who assume that higher hierarchical skills reduce the risk of rework of certain activities in a project. In a lot of papers, one of these described influences of hierarchical skills is also combined with level-specific resource costs. In the hMSRCPSP with a cost minimisation objective, the cost of a multi-skilled resource depends on the number of skills they possess and the level at which they master each skill (Correia and Saldanha-da Gama, 2014; Maghsoudlou et al., 2017; Zabihi et al., 2019; Najafzad et al., 2019; Hosseinian and Baradaran, 2021b). Finally, Snauwaert and Vanhoucke (2022) present various linear programming formulations for a multitude of multi-skilled project scheduling problems with hierarchical skills and propose a modular dataset for these problems or combinations of these problems, which will be studied in the current paper.

3 MSRCPSP

3.1 Problem description

In the MSRCPSP, a project is represented by an acyclic directed graph $G = (N, E)$. The activity set N is a set of nodes that consists of the non-preemptable activities of the project. The activities of set N are indexed in topological order. The dummy activities with index 0 and $|N|-1$ serve as the start and end of the project. The precedence relations are represented by the set E that is a set of edges that specifies the finish-start precedence relations with a time lag of 0 between the activities in N. A set of resources R are assigned to the activities in N in order to execute the project. These resources possess one or more skills of the skill set S, therefore we call them multi-skilled resources. The mastered skills of the resources are called the categorical skills. The addition of categorical skills and multi-skilled resources introduces the following assumptions in this project scheduling problem:

- Each activity $i \in N$ has a predefined standard duration p_i. The dummy activities have duration of 0.
- The skill requirements r_{ij} determine the demand for skills of each activity $i \in N$ for each skill $j \in S$. The dummy activities have no skill demand.
- Resources should be assigned to an activity $i \in N$ such that each individual skill requirement is satisfied.
- Multi-skilled resources are assumed to be human and are renewable.
- Each resource $k \in R$ can only be assigned to one skill per activity in order to assure standard non-changing durations.
- Resources are disjunctive, which means that they cannot work on more than one activity per time unit.
- Resources can master one or more skills. The skill distribution b_{jk} is equal to 1 if resource k masters skill j and 0 otherwise.
- Assigned resources must start simultaneously at an activity

A feasible schedule for the MSRCPSP is created by choosing a start time s_i for each activity $i \in N$ such that the precedence relations are satisfied, and deciding on resource assignments that are within the resource and skill constraints. The following mathematical model is based on the model presented in Correia et al. (2012). The parameters, sets and decision variables utilised in the model below and throughout the rest of this paper are shown in Table 1.

$$\text{Min} \quad s_{|N|} \tag{1}$$

$$\text{s.t.} \quad \sum_{k \in R^j} x_{ijk} = r_{ij} \qquad \forall i \in N^*, j \in S^i \tag{2}$$

$$s_i + p_i \leq s_{i'} \qquad \forall (i, i') \in E \tag{3}$$

$$s_i + p_i - timeUB \cdot (1 - z_{ii'}) \leq s_{i'} \qquad \forall (i, i') \in P \tag{4}$$

$$\sum_{j \in S^{ik}} x_{ijk} \leq 1 \qquad \forall i \in N^k, k \in R \tag{5}$$

$$z_{ii'} + z_{i'i} \leq 1 \qquad \forall (i, i') \in P : i < i' \tag{6}$$

$$\sum_{j \in S^{ik}} x_{ijk} + \sum_{j \in S^{ik}} x_{i'jk} \leq 1 + z_{ii'} + z_{i'i} \qquad \forall i, i' \in N^k \wedge (i, i') \in P, k \in R \tag{7}$$

$$ES_i \leq s_i \leq LS_i \qquad \forall i \in N \tag{8}$$

$$s_i \geq 0 \qquad \forall i \in N \tag{9}$$

$$z_{ii'} \in \{0, 1\} \qquad \forall i, i' \in N \tag{10}$$

$$x_{ijk} \in \{0, 1\} \qquad \forall i \in N, j \in S, k \in R \tag{11}$$

The objective function (1) minimises the start time of the dummy end activity to minimise the makespan of the project. Constraints (2) assure that the skill requirements are satisfied. Constraints (3) & (4) link the activity start times according to the precedence relations. Constraints (5) specify that each

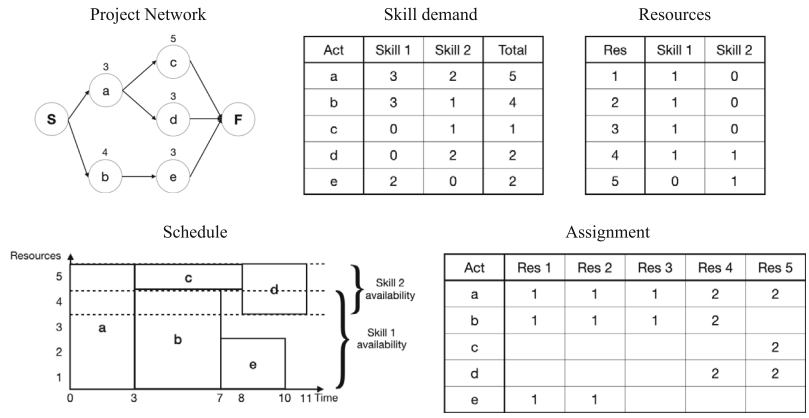
Table 1 Mathematical model details

Parameters	r_{ij}	Number of resources required with skill j by activity i
	p_i	Standard processing time of activity i
	b_{jk}	1, if resource k masters skill j, 0, otherwise
	bl_{jk}	The skill level of resource k at skill j
	$timeUB$	A large number found through a naive method
	r_{ijl}	Number of resources required with skill j and level l by activity i
	c^f/c^f_{jl}	Fixed minimum starting cost and fixed skill cost per skill j and level l
	c^v/c^v_{jl}	Variable minimum starting cost and variable skill cost per skill j and level l
	$\overline{p}$	Deadline on the makespan
	u_{jl}	Usage for a resource with skill j and level l
	u^{rate}	Usage cost rate of a common good
	γ_{jl}	Rework probability of performing skill j with level l
Sets	N	Set of activities
	N^*	$N \setminus \{0, \lvert N\rvert-1\}$
	N^k	Set of activities that require skills that resource k masters
	R	Set of resources
	R^i	Set of resources that master skills required by activity i
	R^{jl}	Set of resources that master skill j at level l
	R^{jl+}	Set of resources that master skill j at level l or higher
	S	Set of skills
	S^i	Set of skills that are required by activity i
	S^{ik}	Set of skills that are required by activity i and can be executed by resource k
	$E \subseteq N \times N$	Set of activity pairs (i, j) such that i directly precedes j
	$P \subseteq N \times N$	Set of activity pairs with no precedence relation
	L	Set of skill levels
	L^{ij}	Set of levels that are required by activity i for skill j
	L^{ijk}	Set of levels that are required by activity i for skill j and can be executed by resource k
Decision variables	x_{ijk}	1, if resource k is assigned to skill j of activity i, 0, otherwise
	$z_{ii'}$	1, if activity i is finished before activity i' starts, 0, otherwise
	s_i	Start time of activity i
	x_{ijkl}	1, if resource k is assigned to skill j and level l of activity i, 0, otherwise
	ω_k	1, if resource k is assigned to any activity in the project, 0, otherwise

resource can only be assigned to one skill in an activity. The logical constraints (6) between the $z_{ii'}$ sequencing variables assure a feasible order between the activities in the project. Constraints (7) ensure that each resource only works on one activity per time unit. Constraints (8) set the lower and upper bound for the start times of the activities. The domain of the decision variables is defined in Constraints (9), (10) & (11).

3.2 Example

Figure 1 displays a comprehensive example for the MSR-CPSP for a project with 5 resources and 2 skills. In the network on the top left, all 5 project activities, 2 dummy activities, the precedence relations and durations are displayed. The table to the right shows the skill demand per activity for skills 1 and 2 and the total number of workers required per activity. For example, activity a requires 3 resources that possess skill 1 and 2 resources that possess skill 2, for a total of 5 required resources. The table with the resource characteristics describes the skills that each of the 5 resources possesses. Resources 1, 2 and 3 master skill 1, resource 4 is multi-skilled, meaning that he/she masters both skills, and resource 5 only masters skill 2. An optimal schedule and assignment are presented at the bottom of this figure. The project starts with activity a, no other activities can be scheduled in parallel due to the restricted amount of resources. In activity a, resources 1, 2 and 3 perform skill 1 and resources 4 and 5 perform skill 2. As activity a finishes at $t = 3$, activity b and c are scheduled to start simultaneously at this time. Resources 1 to 4 are appointed to activity



Act	Skill 1	Skill 2	Total
a	3	2	5
b	3	1	4
c	0	1	1
d	0	2	2
e	2	0	2

Res	Skill 1	Skill 2
1	1	0
2	1	0
3	1	0
4	1	1
5	0	1

Act	Res 1	Res 2	Res 3	Res 4	Res 5
a	1	1	1	2	2
b	1	1	1	2	
c					2
d				2	2
e	1	1			

Fig. 1 Complete example of the MSRCPSP

b and resource 5 to activity c. Activity b finishes at $t = 7$ and activity e is started by resources 1 and 2. Activity d cannot be scheduled yet since no 2 resources are available that can perform skill 2. As such, activity d can only be started after activity c finishes at time $t = 8$. At $t = 11$, activity d is completed and the project is finished.

4 Genetic algorithm

In this section, we will present the basic framework and the components of a genetic algorithm that can solve the hMSRCPSP. This genetic algorithm will then be extended with two types of local searches. Firstly, some versatile local searches are developed that can be implemented for each extension of the hMSRCPSP. These are succinctly described in this section. Secondly, problem-specific local searches are created that can optimally deal with the unique decisions of the extensions. These local searches will be explained, together with the problem descriptions of the extensions, in Sect. 5.

Figure 2 shows a flowchart of the genetic algorithm framework that we will use to solve different extensions of the hMSRCPSP, or MSRCPSP with hierarchical levels of skills. As can be seen from this figure, the main part of each generation in this genetic algorithm is similar for all problems. The most important difference will be the problem-dependent local searches that further optimise the found solutions of the genetic algorithm.

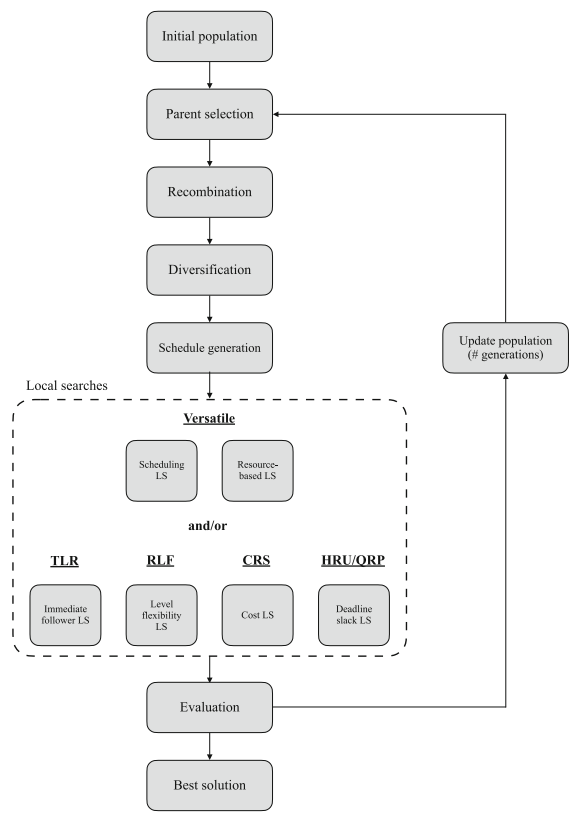


Fig. 2 Flowchart of the genetic algorithm

Representation Each solution in this genetic algorithm is represented by a list divided into two parts. The first $|N|$ values specifies the scheduling order of the activities and the second $|N|$ values determines the assignment priority of the resources. The first $|N|$ values of the representation are made up by an activity list (AL), in which each activity appears once. An activity list provides a sorting in which activities are scheduled. The leftmost activity is considered to have the highest priority. As such, if multiple activities are eligible to be scheduled, the activity that appears most left in the list will be scheduled first. In this list, the activities are ordered in a precedence feasible manner, which means that successors of a certain activity always take place to the right of the activity. Activity lists have been used a lot in the scheduling literature and have been proven to be efficient (Hartmann, 1998).

The second $|N|$ values of the chromosome in this genetic algorithm are part of a priority rule list (PL) (Snauwaert and Vanhoucke, 2021). This a list that assigns an assignment priority rule to each individual activity. Hence, each activity can have a different assignment priority based on the rule that is appointed to it. This can be beneficial because critical activities most likely have other optimal resource assignments than

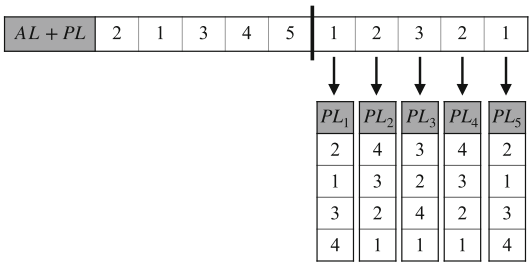


Fig. 3 Example of the representation with the AL and the PL ($|N| = 5$)

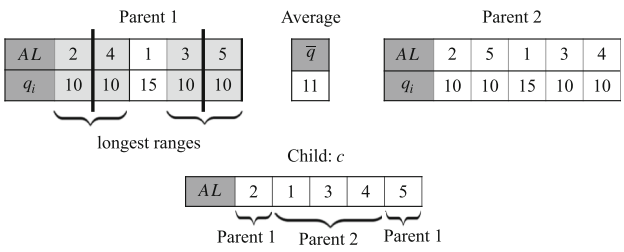


Fig. 4 Illustrative example of the low demand range crossover

non-critical activities, especially taking into account hierarchical skill levels. If an activity is scheduled by the activity list, the PL specifies which priority rule is implemented to assign resources to the activity. If no feasible assignment can be found because the skills of the available resources are not able to satisfy the skill requirements, then the algorithm continues with the next eligible activity from the activity list. The unscheduled activity stays in the activity list and becomes eligible in the next decision moment. The assignment priority rules that are used in this priority rule list are based on categorical skills, hierarchical skills and resource criticality as presented in Snauwaert and Vanhoucke (2021).

Figure 3 displays an example of the representation for a project with 5 activities. The first five values in the list represent the *activity list*, which is the prioritised order in which the algorithm schedules the activities. Specifically, the algorithm will first try to schedule activity 2, then activities 1, 3, 4 and 5. The last five values display the *priority rule list* values. In this case, activities 1 and 5 utilise priority rule 1, activities 2 and 4 utilise priority rule 2 and activity 3 utilises priority rule 3. Below this second list, the priority order of the resources for each of the priority rules is displayed. For example for activity 1, the algorithm will first try to assign resource 2 to a skill requirement, then it will try to assign resources 1, 3 and 4 in that order.

Population and selection At the start of this algorithm, the solution procedure generates an initial population of 75 solutions with a randomly generated AL and PL. This genetic algorithm incorporates elitism; therefore, the two best solutions are always considered to be elite solutions. The parent selection for the recombination step is performed by looping through the complete population to select the first parent and selecting the second parent using a 2-tournament selection procedure.

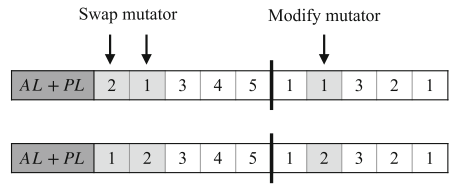


Fig. 5 Illustrative example of the swap and modify mutators

Recombination and diversification The recombination step in this genetic algorithm consists of two crossovers, one of which alters the AL and the other the PL for each pair of parent solutions. The first crossover is a *low demand range crossover* that recombines the activity lists of the selected parents. This crossover is inspired by the peak crossover operator of Valls et al. (2008). This is a 2-point crossover, in which the points are the middle point of the two longest ranges of activities with a lower than average total resource requirement. Two new child solutions are created by copying the activities outside the two points from one parent and filling the activities within the 2-points according to the prioritised order of the other parent. In Fig. 4, the ranges of activities with a lower than average total resource requirement are shown in light grey for parent 1. In this figure, q_i represents the total resource requirement of activity i and $\overline{q}$ is the average total resource requirement. There are two ranges of length 2, which are the two longest ranges. Below, one child solution is shown that is created by copying the activities of parent 1 outside the middle points of these two ranges and filling the activities inside the two points using the order of parent 2. The second crossover is a *random two-point crossover* that is applied to the priority rule lists. This crossover random selects two points in the PL, copies the values outside the two points from one parent and used the order of the other parent for the other activities in the list. The crossover rate set experimentally to 0.8.

In order to create diversification in our solution, the genetic algorithm invokes two mutators. For the AL, a *swap mutator* substitutes two random values in the representation. For the PL, it randomly chooses between a swap mutator and a *modify mutator*. The latter changes the value of one random activity such that it gets assigned resources according to a different resource priority rule. Figure 5 shows an illustrative example that includes the swap mutator on the AL and the modify mutator on the PL. The mutation rate is set experimentally to 0.2.

Schedule generation scheme and population update procedure A parallel schedule generation scheme (PSGS) is executed to generate schedules from the child solutions. This method determines the most prioritised eligible activity at

each decision point. The activity is only scheduled only if the resource constraints, skill constraints and level constraints are satisfied for the problems in Sect. 5. The resources are assigned by use of the priority rule list. After each decision point, the PSGS uses time incrementation to move to the next decision point at which a following activity is scheduled until all activities are scheduled. Of the two created children per set of parent solutions, the best child replaces the first parent. The final step of the update procedure is to update the elite solutions with the two new best found solutions. After 15,000 schedules, the algorithm is truncated.

Versatile local searches Two versatile scheduling and assignment local searches from the literature have been implemented for each of the problems presented in Sect. 5.

As a first local search, we include the iterative scheduling procedure of Li and Willis (1992) in order to further improve the created solutions in each generation. This local search constructs schedules that are left- and right-justified in order to shorten the project schedule without modifying the assignments. This local search is applied to all solutions at each iteration.

The second local search is the resource-based local search of Snauwaert and Vanhoucke (2021), which consists of two main phases. The first phase aims to change resource assignments in the project such that more flexible resources become available. More specifically, for a certain activity this local search identifies which resources are unassigned for the whole duration of the activity. Next, it tries to assign the resources that master the least categorical skills or have the lowest resource criticality to this activity. In the second phase, the local search attempts to modify the schedule by shifting activities to the left. As such, this local search can improve the makespan by improving both the schedule and the resource assignments. Since this local search is less effective for instances with a completely-skilled or a single-skilled workforce, we apply this local search on the 10 best-known solutions for a RA value in]0, 1[at each iteration and on the 5 best-known solutions for the instances with a RA value of 0 and 1.

For all other problem-specific local searches, explained in the next sections, we decided to apply the local search on the 5 best-known solutions on each iteration of the GA based on preliminary tests.

5 MSRCPSP with hierarchical skills

In this section, we will discuss several extensions of the MSRCPSP that consider a different impact of the hierarchical skills. A problem formulation for these problems is given in Snauwaert and Vanhoucke (2022), but no best-known solutions are provided. Table 2 displays the problems that will be investigated in the next sections and their main characteristics. The titles of each section include an abbreviation for each problem, also shown in Column 3, which we will use throughout this paper.

In the problems presented in the next sections, the MSRCPSP is extended with hierarchical skills, which we call the *hMSRCPSP*. The hMSRCPSP specifies the level at which a resource can perform a skill. New assumptions are added to the hMSRCSP due to the inclusion of hierarchical skills:

- A level set L represents the levels at which resources can master the skills in S.
- The skill requirements r_{ij} can be enhanced to skill-level requirements r_{ijl} that also specify the level l at which a skill j is required in an activity i.
- The discrete skill-level distribution bl_{jk} represents the level l at which resource k masters skill j.
- All problems are static in nature, which means that the hierarchical skill levels do not change over time. No learning or training is considered in the problems.
- Each resource k can only perform one skill j and level l in an activity.

In the following sections, we will provide a definition for each problem, explain how a mathematical model can be formed, solve the problem on a small example and describe the developed solution approach that fits in the framework of Sect. 4. Each extension is solved using a hybrid genetic algorithm with the versatile local searches of Sect. 4 and a problem-specific local search that will be discussed separately for each new problem.

5.1 hMSRCPSP with task-level restriction (TLR)

5.1.1 Problem description

The hMSRCPSP with task-level restrictions imposes additional constraints on the activities by specifying that their skill-level requirements r_{ijl} can only be executed by specific resources $k \in R$ that master the level l for skill j. This modification to the MSRCPSP causes the number of possible assignments in the project to be reduced drastically, creating more critical skills and critical levels of skills. This problem is equivalent to the MSRCPSP if for each skill level l of a skill type j in an instance of the TLR problem a new skill type j' is created in an instance of the MSRCPSP and if for the skill-level requirement of that level l a regular skill requirement for the skill j' is added. However, this should not withhold researchers from creating problem-specific procedures for the TLR. Rather, the TLR allows us to solve instances of the MSRCPSP with larger numbers of skill types, which can cause a need for distinct methodological aspects and decision-making. The classification of this problem is $ms,1,H,TR,TR|\text{cpm}|C_{max}$ (Snauwaert and Van-

Table 2 Extension summary table

Section	Problem	Abbreviation
5.1	hMSRCPSP with task-level restriction	TLR
5.2	hMSRCPSP with resource-level flexibility	RLF
5.3	hMSRCPSP with costs of resource skills	CRS
5.4	hMSRCPSP with heterogeneous resource usage	HRU
5.5	hMSRCPSP with quality as rework probability	QRP
5.6	hMSRCPSP with variable activity duration	VAD

houcke, 2023). A mathematical model for the TLR problem can be established from the linear programming formulation of the MSRCPSP by replacing Constraints (2), (5), (7) and (11) for Constraints (12), (13), (14) and (15), shown below.

$$\sum_{k \in R^{jl}} x_{ijkl} = r_{ijl} \qquad \forall i \in N^*, j \in S^i, l \in L^{ij} \tag{12}$$

$$\sum_{j \in S^{ik}} \sum_{l \in L^{ijk}} x_{ijkl} \leq 1 \qquad \forall i \in N^k, k \in R \tag{13}$$

$$\sum_{j \in S^{ik}} \sum_{l \in L^{ijk}} x_{ijkl} + \sum_{j \in S^{ik}} \sum_{l \in L^{i'jk}} x_{i'jkl} \leq 1 + z_{ii'} + z_{i'i}$$
$$\forall i, i' \in N^k \wedge (i, i') \in P, k \in R \tag{14}$$

$$x_{ijkl} \in \{0, 1\} \qquad \forall i \in N, j \in S, k \in R, l \in L \tag{15}$$

In this model, the x_{ijkl} variable is equal to 1 if resource k performs skill j at level l in activity i and 0 otherwise. Constraints (12) guarantee that the skill-level requirements are satisfied. Constraints (13) specify that each resource can only perform one skill at its mastered level in each activity. The disjunctiveness of the multi-skilled resources is assured in Constraints (14). The domain of decision variable x_{ijkl} is expressed in Constraints (15).

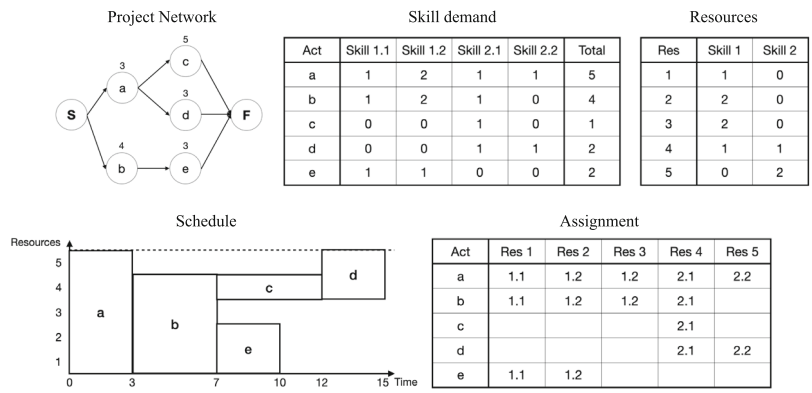
5.1.2 Example

An example for the hMSRCPSP with task-level restriction is shown in Fig. 6, which is an extension of the example of Sect. 3.2. More specifically, the project remains the same, as well as the number of required skills by the activities and the number of mastered skills in the workforce. These last two components are now enhanced with skill levels. For example, activity c now requires 1 resource that masters skill 2 at level 1, and resource 1 masters skill 1 at level 1. The inclusion of the hierarchical skills also changes the assignment which specifies the level at which a skill is performed. A value of 2.1 means that the resource performs skill 2 at level 1. For the TLR problem, the optimal schedule is altered quite drastically. In particular, because activities b and c cannot be performed in parallel any longer, there is only 1 resource that can perform skill 2 at level 1, i.e. resource 4, which severely restricts the scheduling possibilities for this project. As a result, activities a, b, c and d are scheduled in a serial manner, which leads to a project makespan of 15 time units.

5.1.3 Problem-specific local search

For the TLR problem, we have developed an *Immediate Follower Local Search*. This local search looks for resources that are assigned to an activity i and its immediate follower i'. An immediate follower is defined as each activity i' that has a start time $s_{i'}$ equal to the finish time f_i of activity i. Since this procedure attempts to move activity i' earlier in the schedule, it only takes into account the immediate followers that do not have a precedence relation with activity i, i.e. $(i, i') \notin E$. The goal of this local search is to change the assignment of i such that there are no longer overlapping resources, i.e. resources assigned to both activities i and i'. The removal of the overlapping resources then allows the immediate follower i' to be shifted to the left, to schedule it in parallel with activity i. Algorithm 1 presents the pseudocode for this local search which will be described along the following lines. We define two sets of resources: A_i^r is the set of resources assigned to activity i and U_i^r are resources that are idle during the execution of activity i.

For each activity on the critical chain $i \in CC$ and each immediate follower i', this local search verifies whether there are any overlapping resources in their assignments. If so, the procedure releases the resources assigned to activity i. Next, activity i is reassigned using the set of resources A_i^r, previously assigned to activity i, and the unassigned resources U_i^r, from which the local search removes the resources $A_{i'}^r$ assigned to activity i'. Since $A_{i'}^r$ is discarded, we assure that no overlapping resources will be reassigned. If a feasible reassignment is found, the activity i' is shifted to the left within its allowed precedence relations and taking into account the resource constraints. A small illustrative example of this is shown in Fig. 7. In this figure, resource 3 is assigned to both activities. A feasible reassignment can be found for activity i, $A_i^r = \{1, 2, 6\}$, that removes the overlapping resource and allows activity i' to be scheduled in parallel with activity i. This `LeftShift` of activity i' generates a possible `LeftShift` of all activities scheduled after activity i' is finished.



Act	Skill 1.1	Skill 1.2	Skill 2.1	Skill 2.2	Total
a	1	2	1	1	5
b	1	2	1	0	4
c	0	0	1	0	1
d	0	0	1	1	2
e	1	1	0	0	2

Res	Skill 1	Skill 2
1	1	0
2	2	0
3	2	0
4	1	1
5	0	2

Act	Res 1	Res 2	Res 3	Res 4	Res 5
a	1.1	1.2	1.2	2.1	2.2
b	1.1	1.2	1.2	2.1	
c				2.1	
d				2.1	2.2
e	1.1	1.2			

Fig. 6 Complete example of the TLR problem

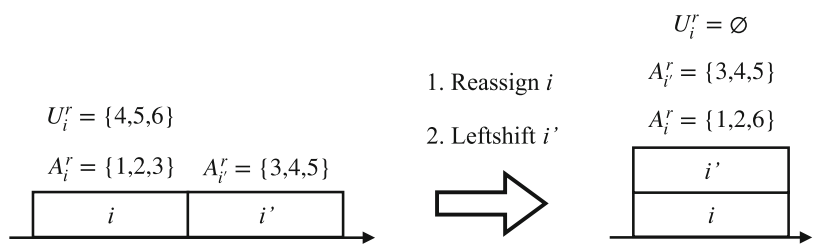


Fig. 7 Example of the reassign and left shift steps

Algorithm 1 Immediate Follower Local Search

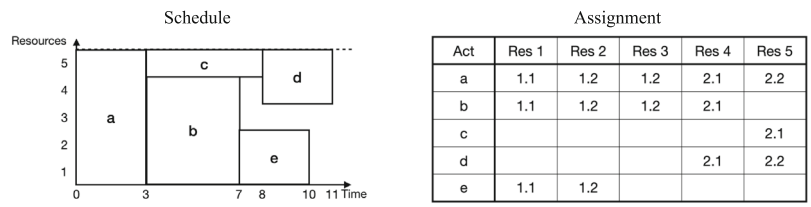
```
for i ∈ AL_1,...,AL_|N| do
  if i ∈ CC then
    for i' ∈ ImmFollower(i, E) do
      if OverlappingRes(x_ijkl, x_i'jkl) then
        U_i^r ← U_i^r ∪ A_i^r
        A_i^r = ∅
        TryReAssign(i,PL_i,U_i^r \ (A_i'^r))
        if FeasibleReAssign() then
          LeftShift(i', x_i'jkl, E)
          for i'' ∈ N : s_i'' ≥ f_i' do
            LeftShift(i'', x_i''jkl, E)
          end for
        end if
        End if
      end if
      End if
    end for
  end if
  End if
end for
```

5.2 hMSRCPSP with resource-level flexibility (RLF)

5.2.1 Problem description

The resource-level flexibility extension of the hMSRCPSP deals with skill-level requirements r_{ijl} that operate as a lower bound for the level of a required skill. In this problem, any resource that masters a skill at the required level or higher can be assigned to the activity. The technological reasoning behind this restriction is that resources who master the skill at a lower level have not had the required training to perform this skill on the activity. In comparison with TLR, the skill-level requirements are less restrictive for the number of possible resource assignments in the projects. This problem is represented as $ms,1,H,TR,Flex|\text{cpm}|C_{\max}$ in the classification of Snauwaert and Vanhoucke (2023).

The RLF problem is rather similar to the TLR problem and only differs in one constraint. More specifically, the R^{jl} set of resources in Constraints (12) needs to be replaced by the R^{jl+} set that includes any resource k mastering skill j at level l or higher. Substituting Constraints (12) by Constraints (16) gives an accurate model for the RLF problem. Constraints (16) stipulate that resources that master skill j at the required level l or higher can be assigned to activity i.



Act	Res 1	Res 2	Res 3	Res 4	Res 5
a	1.1	1.2	1.2	2.1	2.2
b	1.1	1.2	1.2	2.1	
c					2.1
d				2.1	2.2
e	1.1	1.2			

Fig. 8 Example of the RLF problem

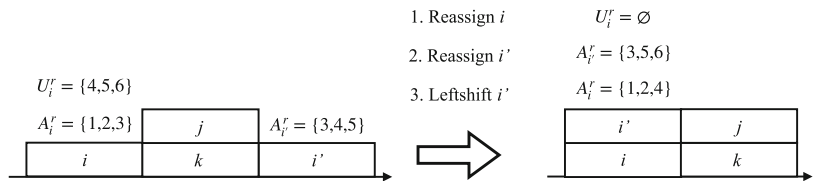


Fig. 9 Example of the reassignment and left shift in the level-flexibility local search

$$\sum_{k \in R^{jl+}} x_{ijkl} = r_{ijl} \qquad \forall i \in N^*, j \in S^i, l \in L^{ij} \tag{16}$$

5.2.2 Example

Figure 8 displays an optimal schedule and assignment for the example in Fig. 6. The increased assignment flexibility of the RLF problem allows for an improved schedule in comparison with Fig. 6. Since resources that master skills with a hierarchical level higher than the required level can also be assigned to a skill, we can now assign resource 5 to both activities b and c. Therefore, if resource 5 is assigned to one of these, activities b and c can be scheduled in parallel, which reduces the total project makespan with 4 time units.

5.2.3 Problem-specific local search

The *level flexibility local search* is designed for the RLF problem. This local search focusses on the increased assignment possibilities due to the new requirement level constraints. More specifically, this method reassigns activities with high-level resources to low-level resources to create left shift opportunities for activities in the schedule. The level flexibility local search is displayed in Algorithm 2. This local search is performed for all activities from 1 to $|N|$. In each iteration, activity i is reassigned with resources that master skills at the lowest hierarchical level. Only idle resources U_i^r and currently assigned resources A_i^r can be assigned. Due to this new assignment, higher-levelled resources might become available to other activities, which can be shifted to the left in parallel with activity i. The `TryReAssignLeftShift` method tries to schedule each activity $i' : s_{i'} \geq f_i$ at $t = s_i$ and assigns resources to activity i' based on the lowest advanced resource criticality rule (LARC) of Snauwaert and Vanhoucke (2021). A low number of categorical skills mastered at a low level coincides with a low advanced resource criticality. If an activity i' is found, its start and finish time is updated and the rest of the schedule is left shifted as much as possible. Figure 9 shows an example of the main steps in this local search. First, activity i is reassigned ($A_i^r = \{1, 2, 4\}$) in order to change the free resources in U_i^r ($= \{3, 5, 6\}$). Then, the procedure checks whether an activity i' can be executed by the resources in U_i^r. If so, this activity i' is reassigned and shifted to the left in parallel with activity i.

5.3 hMSRCPSP with costs of resource skills (CRS)

5.3.1 Problem description

The hMSRCPSP with costs of resource skills replaces the makespan objective with a cost objective using 4 different cost components. A variable ω_k is introduced that is 1 if resource k works on the project and 0 otherwise. Hence, feasible solutions to this problem might only use a subset of the resources in the workforce in order to reduce the total resource cost. Since this problem moves away from the makespan objective, a deadline $\overline{p}$ constraint is added to force a trade-off between variable costs, based on the makespan, and resource and skill cost, based on the assignment. This problem is denoted as *ms,1,H,TR,*{Flex,Cost}|cpm|C (Snauwaert and Vanhoucke, 2023). In the project scheduling literature, the cost of a multi-skilled resource with hierarchical skills is usually characterised by four components. The first component is a *fixed minimum starting cost* c^f, which is a general cost, unaffected by the project makespan, that is equal for all resources as long as they work on a single activity in the project. This fixed minimum starting cost can be perceived as a baseline cost that occurs for a resource to work on the project, disregarding their skills or skill levels. The second component concerns a *fixed skill cost* c_{jl}^f, which is a skill-specific cost that depends on the number of categorical skills a resource masters and the hierarchical level of the mastered skills. The fixed skill cost c_{jl}^f incurs per skill j at level l that a resource masters. The third component is the *variable minimum starting cost* c^v. This cost component takes effect every time unit a resource is working on the

Algorithm 2 Level flexibility local search

```
for i ∈ AL_1,...,AL_|N| do
    Calculate U_i^r
    U_i^r ← U_i^r ∪ A_i^r
    A_i^r = ∅
    TryReAssign(i,U_i^r,Lowest hierarchical level)
    for i' ∈ 1, ..., |N| : s_i' ≥ f_i do
        if TryReAssignLeftShift(i',U_i^r,LARC) then
            Update s_i' and f_i'
            for i'' ∈ N : s_i'' ≥ f_i' do
                LeftShift(i'', x_i''jkl, E)
            end for
        end if
        End if
    end for
end for
```

project. Hence, the total variable minimum start cost for a single resource k can be calculated by multiplying the variable minimum starting cost c^v with the sum of the durations of the activities that the resource k is assigned to. The last cost component is *variable skill cost* c^v_{jl} that is only counted if a resource mastering skill j at level l is working. For example, if a resource k possesses skill 1 at level 2 and skill 2 at level 5 than its total fixed cost is determined by $c^f + c^f_{12} + c^f_{25}$ and its variable cost is calculated as $(c^v + c^v_{12} + c^v_{25}) \cdot x_{ijkl} \cdot p_i$. As a result, the number of categorical skills and the level of hierarchical skills have a positive relation with a resource's cost.

$$\text{Min} \quad \sum_{k \in R} (c^f + \sum_{j \in S^k} \sum_{l \in L} c^f_{jbl_{jk}}) \cdot \omega_k + \sum_{i \in N^*} \sum_{j \in S^i} \sum_{k \in R^{jl+}} \sum_{l \in L} \times (c^v + \sum_{j' \in S^k} c^v_{j'bl_{j'k}}) \cdot x_{ijkl} \cdot p_i \cdot \omega_k \tag{17}$$

$$s_{|N|-1} \leq \overline{p} \tag{18}$$

$$\omega_k \geq x_{ijkl} \qquad \forall i \in N^k, j \in S^{ik}, k \in R, l \in L^{ijk} \tag{19}$$

$$\omega_k \in \{0, 1\} \qquad \forall k \in R \tag{20}$$

The objective function (17) minimises the total fixed and variable resource cost. The parameter bl_{jk} specifies the skill level of resource k for skill j and is equal to 0 if resource k does not master skill j. The fixed skill cost c^f_{jl} and the variable skill cost c^v_{jl} have a value of 0 for level $l = 0$. The deadline of the project is stipulated in Constraints (18). Constraints (19) calculate the value of the resource inclusion variable ω_k. Constraints (20) define ω_k as a binary decision variable.

5.3.2 Example

The example in Fig. 10 addresses the same example project used throughout this paper. In addition to the project network, skill-level requirements and skill-level distribution, the cost information is displayed in the three tables at the top of this figure. In comparison with the solution for the RLF problem, there is a single difference in the optimal solution. In the assignment of b and c, resources 4 and 5 are swapped, i.e. for the CRS problem resource 5 works on activity b and resource 4 executes activity c. Since the skill costs of resource 4 are lower than resource 5, $c^f_{11} + c^f_{21} = 9 < 10 = c^f_{22}$ and $c^v_{11} + c^v_{21} = 0.12 < 0.14 = c^v_{21}$, it is more beneficial to assign resource 4 to activity c because activity c has a longer duration than activity b. Using a cheaper resource for a longer activity reduces the total resource cost, especially because this resource swap does not change the optimal schedule. The optimal resource cost for this small example is equal to 591,92 monetary units.

5.3.3 Problem-specific local search

The local search for the CRS problem aims at reassigning resources to reduce the fixed and/or variable costs of the current schedule. The *Cost local search* minimises the subset of resources that are assigned to the project ($\sum_k \omega_k$) by swapping resource assignments. The steps of this procedure are shown in Algorithm 3. This local search starts by determining the resource $k \in R$ that is assigned the least amount of times over all activities. The procedure enumerates over all activities in order to swap any assignment of resource k with another resource. For each assignment of resource k to an activity i, a sequence of three swaps is attempted.

- The first swap tries to substitute resource k with a resource from the set of unassigned resources U^r_i. Figure 11a shows an example in which resource 3 is swapped with the free resource 6.

Fig. 10 Example of the CRS problem

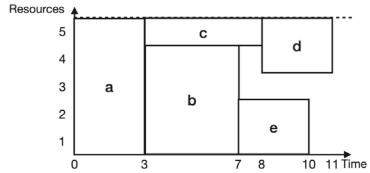
Cost information

Cost	Value
c^f	100
c^v	1

c^f_{jl}	Level 1	Level 2
Skill 1	4	8
Skill 2	5	10

c^v_{jl}	Level 1	Level 2
Skill 1	0.05	0.10
Skill 2	0.07	0.14

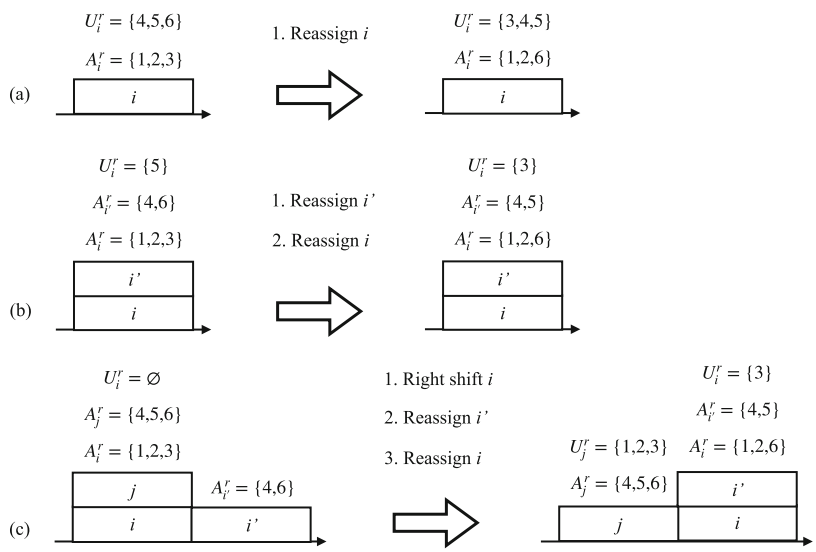
Schedule



Assignment

Act	Res 1	Res 2	Res 3	Res 4	Res 5
a	1.1	1.2	1.2	2.1	2.2
b	1.1	1.2	1.2		2.1
c				2.1	
d				2.1	2.2
e	1.1	1.2			

Fig. 11 Examples of three sequential swaps that remove resource 3



- A more advanced swapping procedure attempts to dismiss resource k by substituting it for a resource k' that is assigned to an activity i' scheduled in parallel to activity i. If there is a feasible candidate k' for this substitution, the local search tries to assign an unassigned resource $k'' \in U_i^r$ in its place to the parallel activity i'. In Fig. 11b, resource 6 of activity i' is substituted with the free resource 5, such that resource 6 can be assigned to activity i for resource 3.
- The third method tries to eliminate the resource k by right shifting activity i without exceeding the deadline. As such, the set of parallel activities of activity i might have changed, which the method utilises to verify whether any other resources can be substituted for resource k. In Fig. 11c, resource 3 cannot be removed from activity i since no free resources are available. Therefore, activity i is right-shifted in parallel with activity i' in order to substitute resource 3 for resource 6 of activity i'.

If any of these methods finds a feasible substitute, the local search moves to the next assignment of resource k until all assignments are dealt with. If neither of the three methods could be executed, the local search is terminated.

5.4 hMSRCPSP with heterogeneous resource usage (HRU)

5.4.1 Problem description

The hMSRCPSP with heterogeneous resource usage deals with the RLF problem under a usage minimisation objective. In this problem, the hierarchical skill levels influence the resource-specific consumption of a renewable or non-renewable common good, e.g. energy or electricity. This common good has a cost rate per period of usage, which needs to be minimised in the objective. The higher the hierarchical skill level of a resource, the more economical the resource can manage the required activity and the lower the total usage will be. The usage u_{jl} is skill and level-specific. As the goal of this problem is to minimise the total cost of usage (Objective (21)), a cost rate parameter u^{rate} that specifies the cost per unit consumed is defined. For example, assume that a resource k that masters skill 1 at level 2 is assigned to an activity i that has a duration p_i equal to 10. If $u_{12} = 20$ and $u^{\text{rate}} = 5$, then the total cost of usage for activity i can be calculated by $u_{12} \cdot u^{\text{rate}} \cdot p_i = 20 \cdot 5 \cdot 10 = 1000$. A deadline constraint is included in this problem in order to cre-

Algorithm 3 Cost local search

```
FindLeastAssignedResource(k ∈ R, x_ijkl)
for i ∈ AL_1,...,AL_|N| do
  if Σ_j Σ_l x_ijkl == 1 then
    TrySwap(k, U_i^r)
    if NoSwap() then
      TrySwap(k, U_i^r ∪ P_i^r)
      if NoSwap() then
        TryRightShiftSwap(k, x_ijkl, U_i^r ∪ P_i^r, E)
        if NoSwap() then
          Stop local Search
        end if
        End if
      end if
      End if
    end if
    End if
  end if
end for
```

ate a trade-off between only assigning the most economical resources to minimise usage and also assigning less economical resources to not exceed the deadline. The HRU problem can be denoted as $ms,1,H,TR,\{\text{Flex,R}\}|\text{cpm}|C$ (Snauwaert and Vanhoucke, 2023).

$$\text{Min} \quad \sum_{i\in N}\sum_{j\in S^{ik}}\sum_{k\in R^i}\sum_{l\in L} x_{ijkl}\cdot u_{jbl_{jk}}\cdot u^{\text{rate}}\cdot p_i \tag{21}$$

5.4.2 Example

Figure 12 exhibits an example of the hMSRCPSP with heterogeneous resource usage. This figure shows an optimal schedule and assignment for the project shown in Fig. 6 and the additional usage information. In this example, the per unit cost of usage u^{rate} is equal to 6 and the usage u_{jl} is between 14 and 24 for the skills and skill levels. An optimal solution to this problem yields the same schedule as the RLF problem. The optimal assignment is also very similar with only one modified assignment. Namely, in activity e the requirement of skill 1 at level 1 is now fulfilled by resource 3 since this resource masters skill 1 at level 2, which results in a lower total usage. The total resource usage cost in this example is equal to 5,004 monetary units.

5.4.3 Problem-specific local search

The *deadline slack local search* delays start times of activities in order to decrease the total usage cost. Algorithm 4 explains the different steps. This local search enumerates over all activities in decreasing order of their duration. For each activity i, the local search tries to reassign the highest levelled resources to decrease the total usage cost. If no reassignment is feasibly possible, the local search right shifts activity i within the deadline slack. The deadline slack is calculated by subtracting the makespan from the deadline. The local search delays activity i iteratively to finishing times of activities later in the schedule. For each shift, the resource with the highest hierarchical skills are assigned to activity i. If this results in an improvement, the remaining schedule is updated and the local search terminates. Otherwise, activity i is further shifted to the next finishing time of a following activity. The local search returns to the original schedule if no improvement is found or the deadline is violated.

5.5 hMSRCPSP with quality as rework probability (QRP)

5.5.1 Problem description

The hMSRCPSP with quality as rework probability minimises the rework probability of the project (Objective (22)). This problem assumes a negative correlation between the rework probability of a resource and its skill level. Resources that master a skill at a higher hierarchical skill level have a lower chance at making mistakes and will therefore have lower rework probability. The rework probability γ_{jl} is a skill level-specific parameter that measures the rework probability if skill j is performed at level l. The constraints of a linear programming formulation for the QRP problem are identical to the constraints of the HRU problem, but the objective is different shown in (22). Despite this difference, the two problems are very similar since it is optimal to assign the resource with the highest hierarchical skill level to each requirement in the project. However, the fact that the usage rate in the HRU problem depends on the durations of the activity causes a difference between the problems. In the HRU problem, the scheduling algorithm should focus more on assigning high-level resources to the longer activities than to the shorter activities, while in the QRP problem the dura-

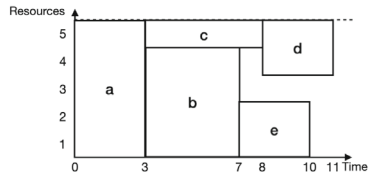
Fig. 12 Example of the HRU problem

Usage information

	Value
u^{rate}	6

u_{jl}	Level 1	Level 2
Skill 1	20	15
Skill 2	24	14

Schedule



Assignment

Act	Res 1	Res 2	Res 3	Res 4	Res 5
a	1.1	1.2	1.2	2.1	2.2
b	1.1	1.2	1.2	2.1	
c					2.1
d				2.1	2.2
e		1.2	1.1		

Algorithm 4 Deadline slack local search

for $i \in N$: $p_i > p_{i+1}$ **do**
  Calculate U_i^r
  $U_i^r \leftarrow U_i^r \cup A_i^r$
  $A_i^r = \varnothing$
  `TryReAssign`*(i,*U_i^r*,Highest hierarchical level)*
  if `NoFeasibleReAssign()` **then**
    `RightShift`*(i,* x_{ijkl}*, E)*
    for $i' \in N : s_{i'} \geq f_i$ **do**
      `RightShift`*(*i'*,* $x_{i'jkl}$*, E)*
    end for
    `TryReAssign`*(i,*U_i^r*,Highest hierarchical level)*
  end if
  End if
end for

tion of the activities has no impact on the final objective value. Snauwaert and Vanhoucke (2023) classifies this problem as *ms,1,H,TR,{Flex,Q}*|cpm|Q.

$$\text{Min} \quad \sum_{i \in N} \sum_{j \in S^{ik}} \sum_{k \in R^i} \sum_{l \in L} \gamma_{jbl_{jk}} \cdot x_{ijkl} \tag{22}$$

5.5.2 Example

Figure 13 shows the rework probability information and the optimal schedule and assignment for the project displayed in Fig. 6. The rework probability γ_{jl} in this figure is between 0.5 and 0.35. Importantly, the rework probability is always lower or equal for higher hierarchical skill levels. The solution of this example is almost identical to the solution for the HRU problem. The schedule is the same, but there is one important difference in the assignment. In the HRU problem, resource 4 is assigned to activity b and resource 5 is assigned to activity c. This is because resource 5 masters skill 2 at a higher level, and therefore, it should be assigned to the longer activity of the two, i.e. activity c, in order to become the optimal solution. Conversely, in the QRP problem, the durations of activities have no impact on the total rework probability. As such, the decision of assigning resources 4 and 5 to activities b and c has no impact on the final solution. The total rework probability for this project is equal to 5.65, divided by the number of assignments; this gives an average activity rework probability of 0.404.

5.5.3 Problem-specific local search

The local search implemented for the hMSRCPSP with quality as rework probability is very similar to the deadline slack local search. The only difference is that it enumerates over the activities following the order of the activity list in this local search. The main principle of the local search remains the same in that it aims to improve the resource assignments such that resources with higher skill levels can be optimally utilised in the project. We refer the reader to Algorithm 4.

5.6 hMSRCPSP with variable duration (VAD)

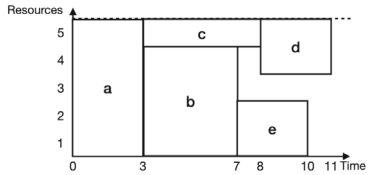
The final problem in this paper deals with hierarchical skills that dictate the speed at which multi-skilled resources perform an activity. A higher skill level induces faster execution speeds and therefore lower activity durations. The duration of activities that require multiple resources and skills is affected by the average skill level of the assigned resources. Following the classification of Snauwaert and Vanhoucke (2023), this problem can be cited as *ms,1,H,TR,Eff*|cpm, *cont*|$C_{\max}$. Since a solution procedure for the hMSRCPSP with variable activity duration that is based on the framework of Sect. 4 was published in Snauwaert and Vanhoucke (2021), we will not explain this problem and its procedure in more detail in this paper. Nevertheless, we include this problem in the computational experiments in order to compare it to the other problems of this section.

Fig. 13 Complete example of the QRP problem

Rework information

γ_{jl}	Level 1	Level 2
Skill 1	0.5	0.35
Skill 2	0.45	0.4

Schedule



Assignment

Act	Res 1	Res 2	Res 3	Res 4	Res 5
a	1.1	1.2	1.2	2.1	2.2
b	1.1	1.2	1.2		2.1
c				2.1	
d				2.1	2.2
e		1.2	1.1		

6 Computational experiments

This section discusses the computational results of the presented solution approaches for each of the problems described in Sect. 5. The experiments are carried out on supercomputer facilities with an Intel Xeon Gold 6140 processor with 2.3 GHz. Section 6.1 reports on the data on which the solution approaches are tested. Section 6.2 presents the results of the presented solution framework and compares it to an IP solver. Section 6.3 analyses the effectiveness of the versatile local searches and the problem-specific local searches. Section 6.4 investigates whether some of the problem-specific local searches are also applicable to an hMSRCPSP they were not initially developed for.

6.1 Dataset design

The data to carry out this research are retrieved from the modular dataset presented in Snauwaert and Vanhoucke (2022). This project dataset is specifically constructed in order to guarantee both realism and diversity in the data. The former is ensured through empirical data validation, while the latter is created through a wide variety of project, resource and skill parameters. Additionally, artificial data are preferred since diversity is hard to assure when solely using empirical data. MSLIB4 of the dataset of Snauwaert and Vanhoucke (2022) is implemented to test the effectiveness of the developed solution framework in this paper. This dataset consists of 'hard' instances, determined by solving all project instances with CPLEX. It is an assembly of 5,000 instances with projects of 30 activities and a broad range of multi-skilled workforces.

6.2 Comparison of solution framework to IP solver

Table 3 compares the results of the genetic algorithm that is extended with the problem-specific local searches (GA-LS) to the results of a continuous-time mathematical formulation (Snauwaert and Vanhoucke, 2022) solved with CPLEX 12.6. The four columns under 'CPLEX (600 s)' present the results of CPLEX as follows: (1) the percentage of instances that are solved to optimality within the time limit of 600 s (%OPT), (2) the percentage of instances for which a feasible non-optimal solution is obtained (%FNO), (3) the fraction of instances that could not be solved (i.e. no feasible solution) within the time limit (%NON) and (4) the average gap percentage (%GAP) of the best found solutions for the instances for which an upper bound was found (%OPT + %FNO). The %GAP in CPLEX is calculated as follows:

$$\%\text{GAP}^{\text{CPLEX}} = (\text{UB}^{\text{CPLEX}} - \text{LB}^{\text{CPLEX}})/\text{UB}^{\text{CPLEX}} \tag{23}$$

The upper bound UB^{CPLEX} is the best integer solution obtained within the time limit of 600 s. The lower bound LB^{CPLEX} is determined by relaxing the integrality constraints of the mathematical formulation.

For the computational results of the solution framework (GA-LS), Table 3 exhibits the average gap percentage (%GAP) of the best found solution and the average CPU time in seconds (s) after 15,000 schedules are generated. This %GAP is calculated by measuring the best found solution of the genetic algorithm UB^{GA} to the lower bound determined by CPLEX LB^{CPLEX}. Comparatively, the %GAP can only be determined for the instances for which a lower bound LB^{CPLEX} was found within the time limit.

$$\%\text{GAP}^{\text{GA}} = (\text{UB}^{\text{GA}} - \text{LB}^{\text{CPLEX}})/\text{UB}^{\text{GA}} \tag{24}$$

Furthermore, Table 3 divides the 5,000 instances of MSLIB4 into distinct subgroups based on their skill strength SS, resource availability RA and average hierarchical level Avg bl_{jk} (Snauwaert and Vanhoucke, 2023). The skill strength SS specifies the skill availability in the multi-skilled workforce. The number of mastered skills in the workforce has a positive relation with the skill strength SS. The resource availability RA determines the resource availability as well as the average number of categorical skills per resource. An instance with a lower RA contains a lower number of resources in the workforce, which means that the skills will be divided over less resources, increasing the average number of categorical skills. Finally, the average hierarchical level Avg bl_{jk} measures the average skill level of all mastered

Table 3 Comparison between solution framework and IP solver for each hMSRCPSP problem

Problem	SS	RA	Avg bl_{jk}	CPLEX (600 s)				GA-LS	
				%OPT	%FNO	%NON	%Gap	%Gap	cpu(s)
TLR	[0;0.5[	[0;0.5[	[0;3]	4.60%	94.73%	0.67%	46.64%	45.57%	44.72
			]3;5]	4.74%	94.79%	0.47%	44.59%	43.51%	44.09
		[0.5;1]	[0;3]	10.15%	89.85%	0.00%	39.69%	38.92%	48.79
			]3;5]	10.26%	89.74%	0.00%	39.13%	38.21%	49.47
	[0.5;1]	[0;0.5[	[0;3]	26.48%	70.93%	2.59%	24.40%	18.60%	52.76
			]3;5]	30.59%	66.47%	2.94%	23.83%	18.12%	53.62
		[0.5;1]	[0;3]	30.61%	69.39%	0.00%	23.64%	23.12%	78.63
			]3;5]	37.93%	62.07%	0.00%	23.01%	21.99%	66.16
RLF	[0;0.5[	[0;0.5[	[0;3]	2.51%	93.14%	4.35%	53.73%	51.77%	48.78
			]3;5]	3.60%	91.94%	4.45%	52.26%	49.93%	48.08
		[0.5;1]	[0;3]	7.41%	91.68%	0.91%	44.88%	44.60%	53.11
			]3;5]	5.40%	92.85%	1.75%	46.43%	45.46%	53.83
	[0.5;1]	[0;0.5[	[0;3]	20.37%	57.41%	22.22%	42.09%	18.76%	57.20
			]3;5]	26.27%	52.16%	21.57%	41.20%	18.40%	58.14
		[0.5;1]	[0;3]	20.41%	65.31%	14.29%	40.38%	27.76%	83.85
			]3;5]	31.03%	55.17%	13.79%	38.8%	25.95%	71.49
CRS	[0;0.5[	[0;0.5[	[0;3]	20.48%	55.85%	23.66%	30.37%	22.80%	37.33
			]3;5]	20.19%	56.78%	23.03%	30.14%	21.41%	36.38
		[0.5;1]	[0;3]	22.58%	49.94%	27.48%	34.90%	30.65%	44.37
			]3;5]	23.21%	49.53%	27.26%	34.84%	29.47%	45.46
	[0.5;1]	[0;0.5[	[0;3]	0.00%	65.74%	34.26%	72.36%	54.02%	52.26
			]3;5]	0.00%	66.27%	33.73%	72.60%	53.99%	53.62
		[0.5;1]	[0;3]	0.00%	57.14%	42.86%	75.31%	57.25%	93.21
			]3;5]	0.00%	72.41%	27.59%	71.16%	54.82%	72.84
HRU	[0;0.5[	[0;0.5[	[0;3]	18.56%	61.87%	19.57%	21.52%	7.69%	66.72
			]3;5]	17.91%	61.23%	20.85%	22.59%	7.47%	65.49
		[0.5;1]	[0;3]	17.10%	58.72%	24.17%	26.45%	13.96%	75.62
			]3;5]	18.22%	55.33%	26.45%	28.32%	13.47%	76.98
	[0.5;1]	[0;0.5[	[0;3]	0.37%	56.67%	42.96%	48.64%	9.11%	88.01
			]3;5]	1.57%	54.51%	43.92%	48.58%	6.89%	89.72
		[0.5;1]	[0;3]	0.00%	67.35%	32.65%	40.33%	13.71%	138.74
			]3;5]	0.00%	68.97%	31.03%	37.27%	7.67%	113.21
QRP	[0;0.5[	[0;0.5[	[0;3]	18.98%	62.46%	18.56%	19.26%	5.57%	60.69
			]3;5]	18.77%	61.04%	20.19%	20.75%	5.92%	59.57
		[0.5;1]	[0;3]	16.19%	57.81%	26.00%	26.74%	12.17%	69.23
			]3;5]	18.08%	56.14%	25.78%	26.43%	9.38%	70.52
	[0.5;1]	[0;0.5[	[0;3]	0.56%	58.33%	41.11%	42.91%	3.04%	80.70
			]3;5]	1.37%	59.80%	38.82%	40.42%	2.40%	82.30
		[0.5;1]	[0;3]	0.00%	59.18%	40.82%	43.16%	5.83%	128.96
			]3;5]	0.00%	68.97%	31.03%	33.15%	1.92%	104.30
VAD	[0;0.5[	[0;0.5[	[0;3]	7.53%	47.58%	44.90%	70.88%	52.56%	21.80
			]3;5]	7.87%	46.35%	45.78%	70.31%	50.38%	21.46
		[0.5;1]	[0;3]	1.60%	44.47%	53.93%	75.83%	44.42%	25.34
			]3;5]	2.43%	44.26%	53.31%	75.30%	43.81%	26.18
	[0.5;1]	[0;0.5[	[0;3]	2.04%	23.15%	74.81%	85.45%	37.49%	31.46
			]3;5]	2.94%	25.10%	71.96%	83.12%	34.44%	32.00
		[0.5;1]	[0;3]	0.00%	10.20%	89.80%	93.78%	19.97%	47.27
			]3;5]	3.45%	10.34%	86.21%	91.38%	22.29%	51.95

Table 4 Analysis for the harder instances with larger CPLEX time limits

Problem	CPLEX (30 m)		CPLEX (1 h)		CPLEX (10 h)		CPLEX (24 h)		GA-LS	
	%NON	%Gap	%NON	%Gap	%NON	%Gap	%NON	%Gap	%Gap	cpu(s)
TLR	2.50%	48.35%	2.50%	47.03%	0.00%	42.60%	0.00%	37.19%	42.62%	62.34
RLF	17.00%	59.71%	15.00%	57.26%	2.00%	46.11%	0.00%	40.32%	45.21%	65.90
CRS	14.00%	37.26%	13.50%	36.22%	2.50%	23.14%	1.00%	18.99%	30.44%	57.03
HRU	16.50%	21.81%	15.50%	20.38%	2.50%	9.44%	1.00%	5.53%	10.53%	81.17
QRP	12.00%	14.70%	11.00%	13.47%	1.50%	6.69%	0.50%	3.60%	8.63%	76.45
VAD	68.00%	88.66%	65.50%	87.22%	53.00%	84.62%	41.50%	83.69%	46.67%	38.46

Table 5 Analysis of the versatile and problem-specific local searches in the solution framework

Setting	TLR		RLF		CRS		HRU		QRP	
	%Gap	cpu(s)	%Gap	cpu(s)	%Gap	cpu(s)	%Gap	cpu(s)	%Gap	cpu(s)
1	37.85%	29.73	43.38%	32.81	33.33%	40.71	12.72%	52.64	9.03%	49.51
2	37.47%	46.72	42.83%	51.77	31.16%	42.22	10.17%	66.00	7.87%	59.32
3	37.41%	45.77	42.32%	52.50	31.24%	43.63	10.01%	62.96	7.26%	58.06
4	37.39%	53.26	41.98%	58.57	29.12%	62.71	9.75%	75.20	6.87%	68.73

skills in the workforce. This value is crucial in this research since it has a direct impact on the studied problems.

Effectiveness of GA-LS: From the values in Table 3, it is clear that the presented solution framework can effectively be applied to hMSRCPSP extensions. More specifically, the %GAP of CPLEX is always higher than the %GAP of the solution framework even though it takes between 5 and 13 times longer to come to these results. The smallest difference between the %GAP of the GA-LS and CPLEX occurs for the TLR problem going from 0.77% up to 5.80% for the subgroups. For higher values of SS, the GA-LS seems to be more effective and the %GAP is lower for both GA-LS and CPLEX. Since the number of assignment possibilities for the TLR problem is lower than for the other problems, the solution space is smaller. Looking at the percentage of instances for which no feasible solution can be found (see Column 7), this leads to a substantially higher percentage of instances that can be feasibly solved for the TLR problem than for the other problems. For the RLF problem, there is a large discrepancy between %GAP differences of the procedures for the two ranges of SS values. The %GAP of our solution framework is considerably lower for the high SS values. The results of the CRS problem show that the solution procedure of Sects. 4 and 5 can efficiently solve the hMSRCPSP with a cost objective. Similar %GAP values can be identified for the HRU and QRP problems. For these problems, the genetic algorithm is clearly more effective, which shows in a large difference in %GAP values. For the VAD problem, there is a more substantial disparity in the %GAP values of CPLEX and the GA-LS. Snauwaert and Vanhoucke (2022) already proved that an IP solver is not able to very effectively solve the VAD problem, due to the nonlinear variable duration equations. Moreover, the number of instances for which no feasible solution was found is greatly affected by the value of the SS and RA.

Impact of resource and skill parameters: For the TLR and RLF problems, more instances are solved to optimality for higher SS values (see Column 5), while for the CRS, HRU and QPR problems the instances with higher SS values appear to be more difficult to solve to optimality. This is due to the fact that the number of mastered skills and their levels directly influence the objective in the latter problems. In the former problems, the number of mastered skills in the workforce impacts the number of assignment possibilities but do not directly impact the makespan objective. Similar conclusions can be reached by analysing the %GAP values for the different ranges of SS. The resource availability RA parameter has lower predictive power. Although the percentage of optimally solved instances is higher for high values of RA, no large differences can be identified. In terms of Avg bl_{jk}, we can state that higher average values of hierarchical levels result in lower %GAP values for both the GA-LS and CPLEX.

Analysis with extended time limit: In order to highlight the power of the GA-LS, we assembled a subset of 200 of the hardest instances from our previous runs by selecting 25 instances for each combination of the three parameters (SS, RA and Avg bl_{jk}) shown in Table 3. To ensure that the 200 instances are complex, they were selected from the MSLIB4 instances that belong to the %NON (no feasible solution) and %FNO (feasible but not optimal) columns of Table 3. Furthermore, from these instances, those with the highest %GAP values were selected, and we then solved them using CPLEX with larger time limits of 30 min, 1 h, 10 h and 24 h.

The results in Table 4 show the fraction of instances that could not be solved within the time limit (%NON) and the %GAP for CPLEX and compares them to the %GAP of the GA-LS for each distinct problem. For each of the 6 problems, we can see an expected steady decrease for the %GAP with the increasing time limit. For the TLR, RLF, HRU and QRP problems, the results of the GA-LS were reached by CPLEX after about 10 h, while the GA-LS achieved these results between 60 and 80 s. For the CRS problem, the 30.44% %GAP of the GA-LS were not attained by CPLEX in 1 h but were significantly surpassed after a time limit of 10 h. Finally, for the VAD problem, even after running CPLEX for 24 h, the results were still far from the results of the GA-LS, clearly showing the complexity of this particular problem.

6.3 Local searches analysis

This section investigates the impact of the versatile and problem-specific local searches on the results of the solution framework. In Table 5, we show the %GAP values for four different settings of the solution framework.

- In setting 1, the genetic algorithm is not extended with any local search.
- Setting 2 specifies the solution approach that is specified in Sect. 4, using two versatile local searches, but without the problem-specific local searches.
- Setting 3 solves the instances using the genetic algorithm with the problem-specific local searches without including the versatile local searches. This setting is incorporated to analyse the direct impact of the problem-specific local searches on the results.
- Setting 4 concerns the complete solution framework with both the versatile and problem-specific local searches.

TLR Both the versatile local search and the "immediate follower local search" do not seem to drastically improve the results for the TLR problem, with a limited improvement of the %GAP of around 0.4%. However, these values are averaged for the complete set of 5,000 instances. Going into more detail regarding the different values of the project, skill and resource parameters, we find that the immediate follower local search is more effective for certain subgroups. For high values of SS the local search performs better. However, since the largest fraction of the instances in MSLIB4 have a lower SS on which the local search has little to no impact, the average impact is rather limited.

RLF For the hMSRCPSP with the RLF extension, the "level flexibility local search" performs better than the versatile local searches. Furthermore, combining both types of local searches leads to the best results (41.98%).

CRS The versatile local searches have a lower average gap (31.16%) than the problem-specific cost local search (31.24%) in the CRS problem when used separately. Nevertheless, the problem-specific "cost local search" is still valuable since it is very effective when combined with the versatile local searches (29.12%). This is because the problem-specific local searches focus on minimising the fixed cost, while the versatile resource-based local search focuses on minimising the variable cost by assigning resources with low advanced resource criticality (ARC) (Snauwaert and Vanhoucke, 2021). Resources with low resource criticality master a low number of resources at a low skill level. Therefore, these are the resources with the lowest variable cost.

HRU/QRP For the HRU and QRP problems, all local searches perform well, as the best setting is always setting 4 that combines both the versatile local searches and the problem-specific "deadline slack local search" (6.87%).

6.4 Impact of problem-specific local searches on other extensions

In the last section of the computational experiments, we investigate how closely the decisions in the hMSRCPSP extensions are related to each other by applying the problem-specific local searches to problems they were not initially designed for. Table 6 displays which local searches we applied to each of the extensions of Sect. 5, and Fig. 14 quantifies the impact in terms of average gap (%GAP). More specifically, we analyse the impact of the immediate follower local search on all problems. The impact of the level flexibility local search is investigated for all problems except for the TLR problem. This is because this local search tries to reassign activities with lower levelled resources, which is not possible in the TLR due to its level requirement restrictions. The cost local search cannot be implemented for the other problems since it solely focuses on fixed costs. Lastly, we test the effectiveness of the deadline slack local search on the CRS problem and the HRU/QRP problems. This local search is not applied to the TLR, RLF and VAD problems, since these problems have no deadline. The problem-specific variable duration local search for the VAD problem, presented in Snauwaert and Vanhoucke (2021), is not applicable to the other problems since it tries to minimise variable durations of individual activities to minimise the makespan.

Figure 14 displays a graph with the average %GAP for the five problem-specific local searches and the procedure without a local search. The problem-specific local search for each hMSRCPSP extension is indicated with an asterisk '*', local searches that could not be applied to a problem are indicated with an 'x'. For the RLF problem, the figure shows that even though the immediate follower local search yields a higher average gap than the problem-specific level flexibility local search, it still improves the solution framework without specific local searches of Sect. 4. For the CRS problem, all the four problem-specific local searches improve

Table 6 Local search and hMSRCPSP combinations

Problem	Immediate follower LS	Level flexibility LS	Cost LS	Deadline slack LS	Variable duration LS
TLR	x				
RLF	x	x			
CRS	x	x	x	x	
HRU	x	x		x	
QRP	x	x		x	
VAD	x	x			x

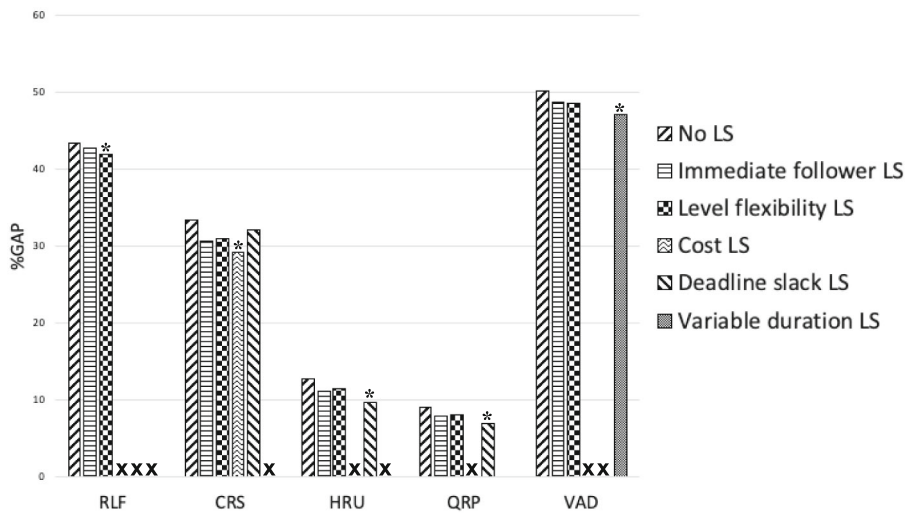


Fig. 14 Comparison between the problem-specific (*) and other applicable local searches for the RLF/CRS/HRU/QRP/VAD problems

the quality of the results. The problem-specific cost local search performs the best with an improvement of 1.5% on the average gap to the second best performing LS. Of the non-problem-specific local searches, the immediate follower local search generates the best results. Nevertheless, all three non-specific local searches have a positive impact on the final objective values for instances of the CRS extension. Finally, for the HRU, QRP and VAD problems, the immediate follower and level flexibility local searches show very similar results. Both the local searches are quite effective in solving the problems; however, they are a lot less effective than the problem-specific local searches. The analysis in Fig. 14 shows that the problem-specific local searches are most effective for their respective problems, however the other local searches can be applied to improve the quality of the results.

6.5 Best known solutions

The results of this paper will be made available to all researchers in a single all-encompassing file on our website (https://www.projectmanagement.ugent.be/research/project_scheduling/MSRCPSP). In this file, we provide the following information and data:

- The file contains results for all 5 MSLIB datasets (Snauwaert and Vanhoucke, 2023), which sums to a total of 32,499 instances
- A separate results page exists for the MSRCPSP and for the 6 hMSRCPSP extensions presented in this paper
- Network parameters, skill parameters, resource parameters and other indicators are provided for each instance.
- A lower bound for each hMSRCPSP extension based on linear relaxation is presented
- The best-known solution of the GA-LS solution framework for all problems is shown for each instance
- The best found solution of CPLEX within the specified time limit of 600 s is provided

The goal of this tool is to contribute to the literature by becoming an important benchmark for future research on the MSRCPSP and hMSRCPSP and allowing fair comparison for other researchers on the presented problems.

7 Conclusion

In this paper, a new solution framework for multi-skilled resource-constrained project scheduling problems with hierarchical skills is proposed. In total, 5 problems are considered

for which problem-specific local searches have been developed. The first problem deals with new task-level restrictions due to the inclusion of hierarchical skills. The second extension allows for higher-levelled resources than required to work on activities. A cost minimisation objective with skill-level-specific costs is introduced in the third problem. Hierarchical skill-level-dependent heterogeneous usage of a good is analysed in the fourth problem. The hMSRCPSP with rework probability minimisation is investigated in the final problem. The computational experiments compare the gathered solutions of the solution framework to an IP solver to prove its effectiveness. We investigate the impact of a versatile scheduling local search and a versatile general resource-based local search in comparison with the problem-specific local searches. This analysis demonstrates that the problem-specific local searches have a positive impact on the results; however, the results are improved when the versatile and problem-specific local searches are combined. Finally, we show that some of the problem-specific local searches can be applied on problems they were not initially developed for; however, this does not always lead to satisfactory results.

Acknowledgements The computational resources (Stevin Supercomputer Infrastructure) and services used in this work were provided by the VSC (Flemish Supercomputer Center), funded by Ghent University, FWO and the Flemish Government - department EWI.

Author Contributions Jakob Snauwaert did conceptualization, methodology, data curation, formal analysis, writing—original draft, and visualisation. Mario Vanhoucke did project administration, resources, supervision, validation, and writing—review & editing.

Funding No funding was received for conducting this study.

Data availability The datasets generated by the survey research during and/or analysed during the current study are available on the website https://www.projectmanagement.ugent.be/research/project_scheduling/MSRCPSP.

Code Availability No

Declarations

Conflict of interest No

Ethical approval Not applicable

Consent to participate Not applicable

Consent for publication Not applicable

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Afshar-Nadjafi, B. (2021). Multi-skilling in scheduling problems: A review on models, methods and applications. *Computers & Industrial Engineering, 151*, 107004.

Almeida, B.F., Correia, I., Saldanha-da Gama, F. (2018). A biased randomkey genetic algorithm for the project scheduling problem with flexible resources. TOP, 1–26.

Almeida, B. F., Correia, I., & Saldanha-da Gama, F. (2019). Modeling frameworks for the multi-skill resource-constrained project scheduling problem: a theoretical and empirical comparison. *International Transactions in Operational Research, 26*(3), 946–967.

Bellenguez, O., & Néron, E. (2004). Lower bounds for the multi-skill project scheduling problem with hierarchical levels of skills. International conference on the practice and theory of automated timetabling (pp. 229–243).

Bellenguez-Morineau, O., & Néron, E. (2007). A branch-and-bound method for solving multi-skill project scheduling problem. *RAIRO-Operations Research, 41*(2), 155–170.

Blazewicz, J., Lenstra, J. K., & Kan, A. R. (1983). Scheduling subject to resource constraints: classification and complexity. *Discrete Applied Mathematics, 5*(1), 11–24.

Brucker, P., Drexl, A., Möhring, R., Neumann, K., & Pesch, E. (1999). Resourceconstrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research, 112*(1), 3–41.

Chen, R., Liang, C., Gu, D., & Zhao, H. (2020). A competence-time-quality scheduling model of multi-skilled staff for it project portfolio. *Computers & Industrial Engineering, 139*, 106183.

Correia, I., Lampreia-Lourenço, L., & Saldanha-da Gama, F. (2012). Project scheduling with flexible resources: formulation and inequalities. *OR spectrum, 34*(3), 635–663.

Correia, I., & Saldanha-da Gama, F. (2014). The impact of fixed and variable costs in a multi-skill project scheduling problem: An empirical study. *Computers & Industrial Engineering, 72*, 230–238.

Dang Quoc, H., Nguyen Doan, C., Xiong, N., et al. (2020). Effective evolutionary algorithm for solving the real-resource-constrained scheduling problem. Journal of Advanced Transportation, 2020 .

De Bruecker, P., Van den Bergh, J., Beliën, J., & Demeulemeester, E. (2015). Workforce planning incorporating skills: State of the art. *European Journal of Operational Research, 243*(1), 1–16.

Demeulemeester, E. L., & Herroelen, W. S. (2006). *Project scheduling: a research handbook* (Vol. 49). Berlin: Springer Science & Business Media.

Drezet, L.-E., & Billaut, J.-C. (2008). A project scheduling problem with labour constraints and time-dependent activities requirements. *International Journal of Production Economics, 112*(1), 217–225.

Eiselt, H. A., & Marianov, V. (2008). Employee positioning and workload allocation. *Computers & Operations Research, 35*(2), 513–524.

Haroune, M., Dhib, C., Neron, E., Soukhal, A., Babou, H.M., Nanne, F.M. (2022). A hybrid heuristic for a two-agent multi-skill resourceconstrained scheduling problem. International Journal of Advanced Computer Science and Applications, 13 (5).

Hartmann, S., & Briskorn, D. (2021). An updated survey of variants and extensions of the resource-constrained project scheduling problem. European Journal of Operational Research.

Hartmann, S. (1998). A competitive genetic algorithm for resource-constrained project scheduling. *Naval Research Logistics (NRL), 45*(7), 733–750.

Hartmann, S., & Briskorn, D. (2010). A survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research, 207*(1), 1–14.

Herroelen, W., Demeulemeester, E., De Reyck, B. (1999). A classification scheme for project scheduling. Project scheduling (pp. 1–26). Springer.

Hosseinian, A. H., & Baradaran, V. (2021). A multi-objective multi-agent optimization algorithm for the multi-skill resource-constrained project scheduling problem with transfer times. *RAIRO-Operations Research, 55*(4), 2093–2128.

Hosseinian, A. H., & Baradaran, V. (2021). A two-phase approach for solving the multi-skill resource-constrained multi-project scheduling problem: a case study in construction industry. *Engineering, Construction and Architectural Management, 30*(1), 321–363.

Javanmard, S., Afshar-Nadjafi, B., & Niaki, S. T. A. (2017). Preemptive multiskilled resource investment project scheduling problem: Mathematical modelling and solution approaches. *Computers & Chemical Engineering, 96*, 55–68.

Javanmard, S., Afshar-Nadjafi, B., & Taghi Akhavan Niaki, S. (2022). A biobjective model for scheduling of multiple projects under multi-skilled workforce for distributed load energy usage. *Operational Research, 22*(3), 2245–2280.

Laszczyk, M., & Myszkowski, P. B. (2019). Improved selection in evolutionary multi–objective optimization of multi–skill resource–constrained project scheduling problem. *Information Sciences, 481*, 412–431.

Li, Y.-Y., Lin, J., & Wang, Z.-J. (2022). Multi-skill resource constrained project scheduling using a multi-objective discrete jaya algorithm. *Applied Intelligence, 52*(5), 5718–5738.

Lin, J., Zhu, L., & Gao, K. (2020). A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. *Expert Systems with Applications, 140*, 112915.

Li, K. Y., & Willis, R. J. (1992). An iterative scheduling technique for resourceconstrained project scheduling. *European Journal of Operational Research, 56*(3), 370–379.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2016). A multi-objective invasive weeds optimization algorithm for solving multi-skill multi-mode resource constrained project scheduling problem. *Computers & Chemical Engineering, 88*, 157–169.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2017). Multi-skilled project scheduling with level-dependent rework risk; three multiobjective mechanisms based on cuckoo search. *Applied Soft Computing, 54*, 46–61.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2019). Preemptive multiskilled resource constrained project scheduling problem with hard/soft interval due dates. *RAIRO-Operations Research, 53*(5), 1877–1898.

Maghsoudlou, H., Afshar-Nadjafi, B., & Niaki, S. T. A. (2021). A framework for preemptive multi-skilled project scheduling problem with time-of-use energy tariffs. *Energy Systems, 12*(2), 431–458.

Montoya, C., Bellenguez-Morineau, O., Pinson, E., & Rivreau, D. (2014). Branch-and-price approach for the multi-skill project scheduling problem. *Optimization Letters, 8*(5), 1721–1734.

Myszkowski, P. B., & Laszczyk, M. (2022). Investigation of benchmark dataset for many-objective multi-skill resource constrained project scheduling problem. *Applied Soft Computing, 127*, 109253.

Myszkowski, P. B., Olech, L. P., Laszczyk, M., & Skowrónski, M. E. (2018). Hybrid differential evolution and greedy algorithm (degr) for solving multi-skill resource-constrained project scheduling problem. *Applied Soft Computing, 62*, 1–14.

Myszkowski, P. B., Skowroński, M. E., Olech, ŁP., & Oślizło, K. (2015). Hybrid ant colony optimization in solving multi-skill resource-constrained project scheduling problem. *Soft Computing, 19*(12), 3599–3619.

Najafzad, H., Davari-Ardakani, H., & Nemati-Lafmejani, R. (2019). Multi-skill project scheduling problem under time-of-use electricity tariffs and shift differential payments. *Energy, 168*, 619–636.

Néron, E. (2002). Lower bounds for the multi-skill project scheduling problem. In Proceeding of the eighth international workshop on project management and scheduling (pp. 274–277).

Polo-Mejía, O., Artigues, C., Lopez, P., & Basini, V. (2019). Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility. *International Journal of Production Research, 58*(23), 7149–7166.

Saxena, A. (2014). Workforce diversity: A key to improve productivity. *Procedia Economics and Finance, 11*, 76–85.

Snauwaert, J., & Vanhoucke, M. (2021). A new algorithm for resource-constrained project scheduling with breadth and depth of skills. *European Journal of Operational Research, 292*(1), 43–59.

Snauwaert, J., & Vanhoucke, M. (2022). Mathematical formulations for project scheduling problems with categorical and hierarchical skills. *Computers & Industrial Engineering, 169*, 108147.

Snauwaert, J., & Vanhoucke, M. (2023). A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem. *European Journal of Operational Research, 307*(1), 1–19.

Tian, Y., Xiong, T., Liu, Z., Mei, Y., & Wan, L. (2022). Multi-objective multi-skill resource-constrained project scheduling problem with skill switches: Model and evolutionary approaches. *Computers & Industrial Engineering, 167*, 107897.

Tiwari, V., Patterson, J. H., & Mabert, V. A. (2009). Scheduling projects with heterogeneous resources to meet time and quality objectives. *European Journal of Operational Research, 193*(3), 780–790.

Valls, V., Ballestin, F., & Quintanilla, S. (2008). A hybrid genetic algorithm for the resource-constrained project scheduling problem. *European Journal of Operational Research, 185*(2), 495–508.

Valls, V., Pérez, Á., & Quintanilla, S. (2009). Skilled workforce scheduling in service centres. *European Journal of Operational Research, 193*(3), 791–804.

Wang, L., & Zheng, X.-L. (2018). A knowledge-guided multi-objective fruit fly optimization algorithm for the multi-skill resource constrained project scheduling problem. *Swarm and Evolutionary Computation, 38*, 54–63.

Yannibelli, V., & Amandi, A. (2011). A knowledge-based evolutionary assistant to software development project scheduling. *Expert Systems with Applications, 38*(7), 8403–8413.

Yu, Y., Xu, Z., Liu, D., & Zhao, S. (2022). A two-stage approach with softmax scoring mechanism for a multi-project scheduling problem sharing multiskilled staff. *Expert Systems with Applications, 203*, 117385.

Zabihi, S., Kahag, M. R., Maghsoudlou, H., & Afshar-Nadjafi, B. (2019). Multi-objective teaching-learning-based meta-heuristic algorithms to solve multi-skilled project scheduling problem. *Computers & Industrial Engineering, 136*, 195–211.

Zheng, H.-Y., Wang, L., & Zheng, X.-L. (2017). Teaching–learning-based optimization algorithm for multi-skill resource constrained project scheduling problem. *Soft Computing, 21*(6), 1537–1548.

Zhu, L., Lin, J., Li, Y.-Y., & Wang, Z.-J. (2021). A decomposition-based multiobjective genetic programming hyper-heuristic approach for the multiskill resource constrained project scheduling problem. *Knowledge-Based Systems, 225*, 107099.

Zhu, L., Lin, J., & Wang, Z.-J. (2019). A discrete oppositional multi-verse optimization algorithm for multi-skill resource constrained project scheduling problem. *Applied Soft Computing, 85*, 105805.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.